

dnt API Manual

None

None

None

Table of Contents

1. dnt API Manual	3
2. Quickstart	4
3. Changelog	5
3.1 0.3.3 — 2026-09-26	5
4. API Reference	8
4.1 API Reference	8
4.2 dnt Package	9
4.3 Detection	0
4.4 Tracking	0
4.5 Engine	0
4.6 Filter	0
4.7 Labeler	0
4.8 Shared Utilities	0

1. dnt API Manual

This site is generated with MkDocs and `mkdocstrings` from the Python source in `src/dnt`.

Use the navigation to browse package modules and APIs.

2. Quickstart

1. Install the package in your environment.
2. Run one of the examples in `examples/`.
3. Open the API reference for detailed class and function docs.

3. Changelog

3.1 0.3.3 — 2026-09-26

3.1.1 Important

- **Tracker settings now take effect.** In 0.3.2.x every tuning field of the tracker configs was silently ignored and BoxMOT's defaults were used. Settings now reach BoxMOT. Configs you never tuned keep their results: with the dependency versions in `tests/golden/reference-env.txt`, untuned tracks are unchanged from 0.3.2.4. Tuning fields now default to `None`, meaning "BoxMOT's value".
- Tracker configs are keyword-only: `BoTSORTConfig(ReIDWeights.X)` now raises `TypeError` (it previously set `model` by mistake). Use `BoTSORTConfig(reid_weights=ReIDWeights.X)`.
- Settings that cannot affect the chosen tracker raise `ValueError`: SF-SORT `det_thresh`, `max_age`, `min_hits`, `iou_threshold`, `asso_func`; BoostTrack `asso_func`. SF-SORT values outside the range BoxMOT would silently clamp also raise.
- `extra_kwargs`: 0.3.2.x factory keys that worked (`evolve_param_dict`, `tracker_config`, `per_class`, `reid_weights` on ReID trackers, and a `tracker_type` override on an untuned config) still work with identical results and are deprecated for 0.4. A `tracker_type` override combined with the source config's own tuning fields (which 0.3.2.4 discarded) now raises. Keys that were silently ignored now raise; other tracker parameters are validated like fields.
- Saved configs: `from_dict()`, `import_yaml()` and `Tracker(config_yaml=...)` recognise 0.3.2.x output (YAML files and `to_dict()` dicts) and migrate it — untouched defaults follow BoxMOT (results unchanged), changed values take effect. Limitation: a value deliberately set equal to the old dnt default also migrates; the load warning lists every such field. 0.3.3 mappings carry `dnt_config_version: 2`. `XConfig(**old_dict)` is not migrated (dnt warns when it sees that pattern); use `from_dict`.
- `Synchronizer`: timestamps now continue across videos and `offsets` are applied (formula in the docstring); results change for every video after the first and wherever offsets are non-zero. A non-zero `offsets[0]` raises `ValueError`.
- Golden comparison: with the reference environment, untuned tracks are byte-identical to 0.3.2.4 for ByteTrack, BoT-SORT, StrongSORT, BoostTrack and SF-SORT and the `evolve_param_dict` passthrough. OC-SORT, Deep OC-SORT, HybridSORT and a `tracker_type` override to OC-SORT are exempt (reviewed defects BX16-NP2-*): they crash identically on 0.3.2.4 and 0.3.3 (see Known issues).
- `Tracker(device=...)` now uses the same device names as `Detector`: `auto`, `cpu`, `cuda`, `cuda:N`, `mps`, `xpu[:N]`. BoxMOT-style bare indices such as `device="0"` now raise `ValueError` (use `"cuda:0"`); `device="cuda"` now works. With `auto`, trackers use CUDA if available, then Apple MPS, else CPU (XPU hosts track on CPU, as before) — on Apple Silicon, ReID trackers now run on MPS instead of CPU.
- **Detector now defaults to the batched fast path.** It gains `fast` (default `True`) and `batch` (default `8`), read by `detect()` (and, through it, `detect_batch()`) on every call: with `self.fast` `True`, `detect()` redirects to `detect_fast(..., batch=self.batch)` instead of running its own per-frame pipeline — see `detect_fast()`'s changelog entry below for what that changes numerically (exact at `batch=1`; small, bounded, expected drift at `batch>1` with `half=True`, from FP16 batching numerics, not a defect). Existing code that constructs `Detector(...)` without `fast=/batch=` is now on this path by default for every `detect()/detect_batch()` call. Pass `Detector(fast=False, ...)`, or set `detector.fast = False` after construction, to keep the exact previous per-frame behavior for later calls (required for `show=True`, which `detect_fast()` doesn't support — calling `detect(show=True)` while `self.fast` is `True` raises `ValueError`). `fast` / `batch` are plain, mutable instance attributes, not `detect()` / `detect_batch()` call arguments — mirroring `conf`, `nms`, `device` and the rest of `Detector`'s existing settings.

3.1.2 Added

- `Detector` gains `imgsz`, `classes`, `rect`, `agnostic_nms` and `embed`, forwarded to Ultralytics `model.predict()` in both `detect()` and `detect_frames()`. They default to Ultralytics' own defaults (`640`, `None`, `False`, `False`, `None`), so existing calls are unaffected.
- `Detector` gains `class_names`, sugar for `classes`: resolves class names (e.g. `["car", "truck"]`) against `dnt.shared.util.load_class_dict()` (COCO names) and merges them into `classes`. Raises `ValueError` for an unknown name.
- `Tracker.track()/track_batch()` gain `verbose` (default `True`), matching `Detector.detect()`. `verbose=False` disables the per-video progress bar.

- `StrongSORTConfig` gains `min_conf`. BoxMOT's StrongSORT drops every detection scored below it before association, and the value it gets by default (from BoxMOT's YAML) is `0.6`, a hard cut with no low-score second stage. It could not be changed before; e.g. `StrongSORTConfig(min_conf=0.3)` now keeps weaker detections. The default (`None`) keeps `0.6`, so untuned results are unchanged.
- `Filter.deduplicate_boxes(detections, iou_thresh=0.45, containment_thresh=0.65)`: suppresses duplicate/nested-sub-box detections within each frame (e.g. a torso box nested inside a full-body box), keeping the higher-confidence box of each overlapping pair. Built on `dnt.engine.iou()`; both the IoU and containment checks it uses follow `cython_bbox.bbox_overlaps()`'s pixel-inclusive area convention ($(w+1)*(h+1)$), consistent with the rest of dnt's IoU-based matching.
- `Labeler.draw_tracks()` gains `message` (default `""`), matching `Detector.detect()/Tracker.track()`. It sets the progress-bar text shown after the video/batch position; `None` falls back to `input_video`, same as `Tracker.track()`'s fallback to the video file name. Ignored when `compress_message` is `True`.
- `Detector.detect_fast(input_video, ..., batch=None)`: same output as `detect()`, but batches `batch` (or `self.batch` if not given; see `Detector`) frames through the underlying Ultralytics `AutoBackend` in one forward pass instead of calling `model.predict()` once per frame. `detect()`'s per-frame path rebuilds an inference dataset and forces several CUDA synchronizations on every single-frame call, regardless of batch size; measured on an RTX 5070 Ti with RT-DETR-x, this is roughly 3-4x faster. At `batch=1` it reproduces `detect()` exactly (verified frame-for-frame, CPU and CUDA, YOLO and RT-DETR — see `detect_fast()`'s tests and docstring); at `batch>1` on CUDA with `half=True`, FP16's batched cuDNN/cuBLAS kernels can shift a small fraction of results (measured: ~6% of frames, vs ~2.5% for `half=False`), which is inherent to batched half-precision GPU inference and not specific to this method. No `show=` preview (batching trades per-frame latency for throughput); use `detect()` for that.
- `Detector.detect()` and `detect_fast()` gain `return_df` (default `True`). With `return_df=False`, detections are only written to `iou_file` (then required) and `None` is returned. For a 24-hour video (~13-22 M detections) this avoids holding every row in memory.

3.1.3 Changed

- `detect_fast()` (and so `detect()` by default) now writes `iou_file` as it goes: each batch's rows are appended to `<iou_file>.part`, which is renamed to `iou_file` only once the whole range is done, so an existing `iou_file` is always complete. The finished file is byte-for-byte what the previous single write produced. An interrupted run leaves the `.part` file, and the next call with the same `iou_file` resumes from it (dropping and re-detecting its last frame, which may have been cut off); delete the `.part` file to start over. Rows are also kept as NumPy arrays rather than one dict per box (~64 vs ~431 bytes each), so memory stays far lower even with `return_df=True`.
- `detect_batch()` no longer builds the returned table it discards (it passes `return_df=False` when writing files), and with `is_overwrite=True` it deletes a video's stale `.part` file instead of resuming from it.
- `Tracker.track()/track_batch()` `message` now defaults to `""`, so the progress bar no longer shows the video file name (Tracking 1 of 3 instead of Tracking 1 of 3 - <file name>). Pass `message=None` to get the file name back, or any string to show that instead.
- `Detector.detect()/detect_batch()` no longer print Wrote detections to <file> after writing a detection file. `detect_batch()` still returns the written file paths.
- `Labeler.draw_tracks()`'s progress bar no longer shows `input_video` by default outside of batch mode (Generating labels instead of Generating labels <file>), matching `Tracker.track()`'s `message=""` default. Pass `message=None` for the previous behavior. Its batch-mode check (`video_index/video_tot`) also now uses `is not None`, like `Tracker.track()`, instead of a truthy check, so `video_index=0` is now recognized as being in a batch.

3.1.4 Fixed

- `interpolate_tracks_rts(track_file=...)` no longer drops the first row (B3).
- `Filter.interpolate_tracks_rts` uses the package module instead of loading a second copy (B4).
- `Filter.filter_iou` works with zones (Polygon, MultiPolygon or list) (B5).
- `ReClass()` works with its defaults; `Detector(model=...)` accepts names such as `"rtdetr-x"` (B6).
- FP16 is used on indexed CUDA devices (`cuda:0`); the tracker only uses half precision on CUDA (B8).
- `detect_frames` rounds like `detect` (B9).

- `Labeler.draw_dets` accepts `Detector.detect()` output (B10).
- `SignalDetector` handles a short last batch (B13), no longer downloads ImageNet weights and loads on CPU-only machines.
- `Labeler` releases video captures; `export_track_frames(bbox=False)` writes frames (B14).
- `Tracker.track` no longer re-runs `update()` after a `TypeError`; empty detection files produce an empty track file.
- `track_batch` output names strip only a trailing `_iou`.
- `Detector` and `Segmentor` no longer pass `half=` to Ultralytics `model.predict()` on Ultralytics versions that deprecated it in favor of `quantize=` — that combination logged a `WARNING` on every single frame. `dnt._device.predict_precision_kwargs()` detects which the installed Ultralytics accepts and passes the matching kwarg; `Detector(half=...)` / `Segmentor(enable_half=...)` themselves are unchanged.
- `Filter.filter_iou()` and `Filter.deduplicate_boxes()` raised `KeyError` when passed the DataFrame `Detector.detect()` returns directly (named `Detector.DET_FIELDS` columns), since both were written for the positional/headerless layout (`pd.read_csv(file, header=None)`). Both now accept either layout and preserve whichever one the input had.

3.1.5 Packaging

- Requires Python 3.11+ (the code already did).
- `boxmot==16.0.11` pinned (0.3.2.4 allowed BoxMOT 25, whose API breaks tracking).
- The class-name files (`coco/openimages/voc.names`) now ship in the wheel (B7).
- The default ReID weight (`osnet_x1_0_msmt17.pt`, used by `Tracker()`'s default BoT-SORT) and the pedestrian-signal weight (`ped_signal.pt`) now ship in the wheel, so a clean install no longer downloads them on first use.
- Removed dependencies `dnt` never imports — install them yourself if you relied on them: `torchaudio`, `opencv-contrib-python`, `faiss-cpu`, `faiss-gpu-cu12`, `cython`, `easydict`, `h5py`, `motmetrics`, `ninja`, `numpy_indexed`, `prettytable`, `pycocotools`, `scikit-image`, `shapelysmooth`, `sympy`, `tabulate`, `tensorboard`, `termcolor`, `thop`, `vidgear`, `openpyxl`, `matplotlib`, `gdown`, `lapx`, `loguru`, `yacs`, `scikit-learn`.
- Top-level module names such as `detector`, `shared` and `filter` are no longer importable (they were side effects of `sys.path` manipulation, never public API).

3.1.6 Deprecated

- `StrongSORTConfig(max_dist=...):` use `max_cos_dist`.
- `extra_kwargs` keys `evolve_param_dict`, `tracker_config`, `per_class`, `reid_weights`, `tracker_type`, `device`, `half` (removed in 0.4).

3.1.7 Known issues

- OC-SORT, Deep OC-SORT and HybridSORT (including `extra_kwargs["tracker_type"]` overrides targeting them) raise `TypeError: only 0-dimensional arrays can be converted to Python scalars` as soon as a lost track is re-detected. This is a BoxMOT 16.0.11 defect in `unfreeze()` under `numpy >= 2` and was already present in 0.3.2.x. Use ByteTrack, BoT-SORT, StrongSORT, BoostTrack or SF-SORT until the BoxMOT upgrade planned for `dnt` 0.4. Building one of these trackers now emits a `UserWarning`, and the `TypeError` carries a note explaining the issue; the exception itself (type, message) is unchanged.
- `extra_kwargs["tracker_type"]` overrides from a non-ReID config (ByteTrack, OC-SORT, SF-SORT) to a ReID tracker now raise `ValueError`; in 0.3.2.4 all five such combinations crashed inside BoxMOT.

4. API Reference

4.1 API Reference

The pages in this section are rendered from module docstrings and object signatures.

4.2 dnt Package

DNT package for detection and tracking.

This package provides detection and tracking functionality.